Introduction to ASP.NET Core Framework

In today's world of software development, developers seek tools that are **free**, **fast**, **open-source**, and compatible **across various platforms,** including Windows, Linux, and macOS. Microsoft, known for building Windows-based solutions, took a significant step forward by creating **ASP.NET Core**, a modern web framework that lets developers build robust and scalable web applications.

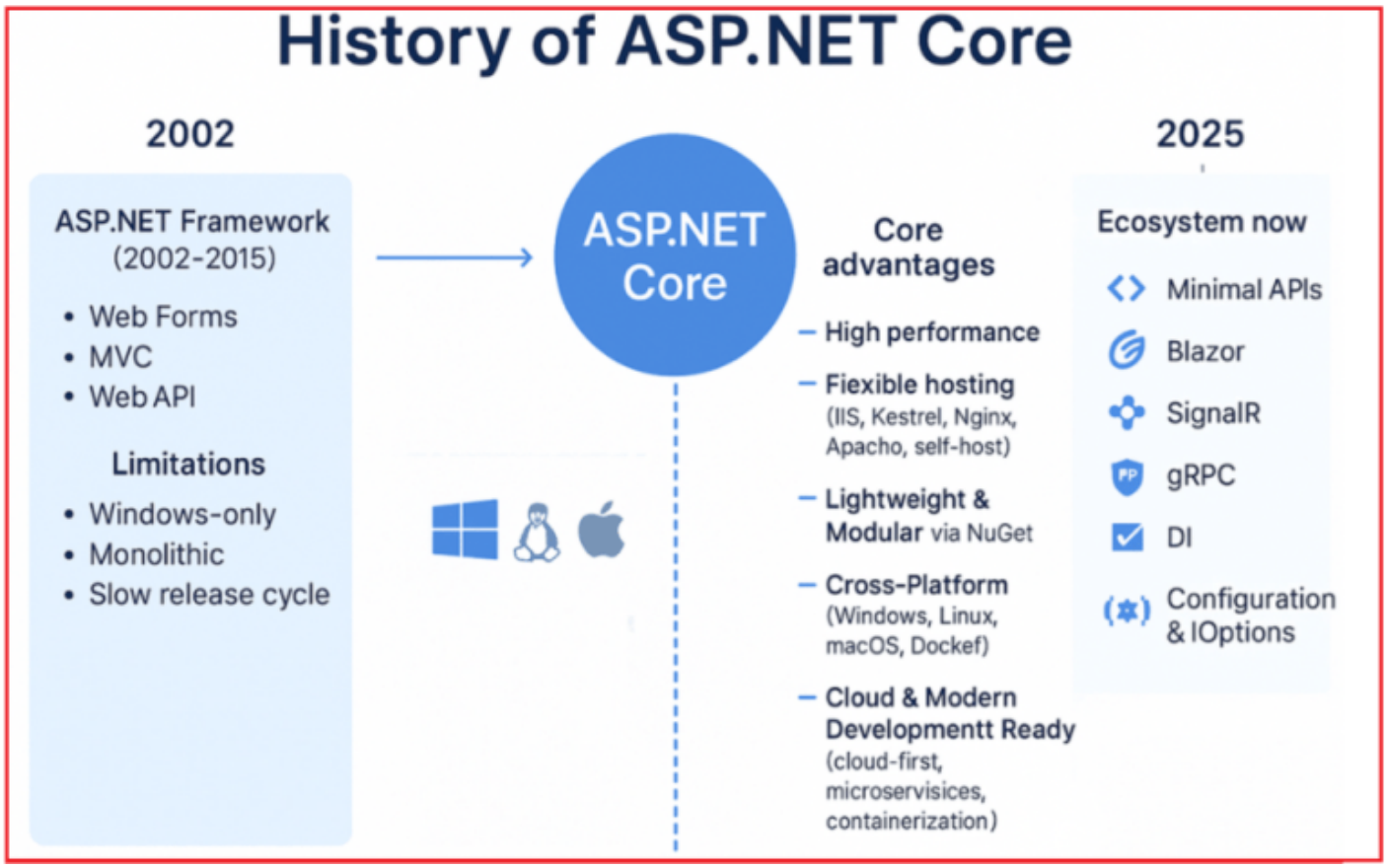
Unlike the old ASP.NET Framework, which was limited to Windows, **ASP.NET Core is cross-platform**, lightweight, and designed for modern cloud-based development. Whether you're building APIs, websites, or microservices, ASP.NET Core gives you the tools to do it faster, better, and with more flexibility.

History of ASP.NET Core

For many years, **ASP.NET Framework** (launched in 2002) was the primary choice for developers to build **data-driven web applications** on the Windows platform. It powered millions of websites and enterprise solutions worldwide. It provided features like **Web Forms, MVC, and Web API**, and was tightly integrated with **Internet Information Services (IIS)**. Over time, however, the **ASP.NET Framework** began to show limitations:

- It was **Windows-only**, making cross-platform development impossible.
- Its **monolithic architecture** meant applications carried many features they might not need, reducing efficiency.
- The release cycle was slower, and updates required waiting for significant framework updates.
- Not well-suited for the new era of **cloud computing, microservices, and cross-platform development**.

For a better understanding, please have a look at the following image.



To overcome these challenges, Microsoft introduced **ASP.NET Core in 2016** as a **complete redesign**, not a continuation of the old framework. Unlike the ASP.NET Framework, which was tied to IIS and Windows, ASP.NET Core was built with a **modular, lightweight, and cross-platform architecture**. This means developers can now run applications on **Windows, Linux, macOS, and inside Docker containers** without being restricted to a single platform.

Fundamental changes in ASP.NET Core

While ASP.NET Core retains familiar concepts like **Controllers, Razor Views, and Routing**, it introduces **fundamental changes** in how applications are built, deployed, and scaled:

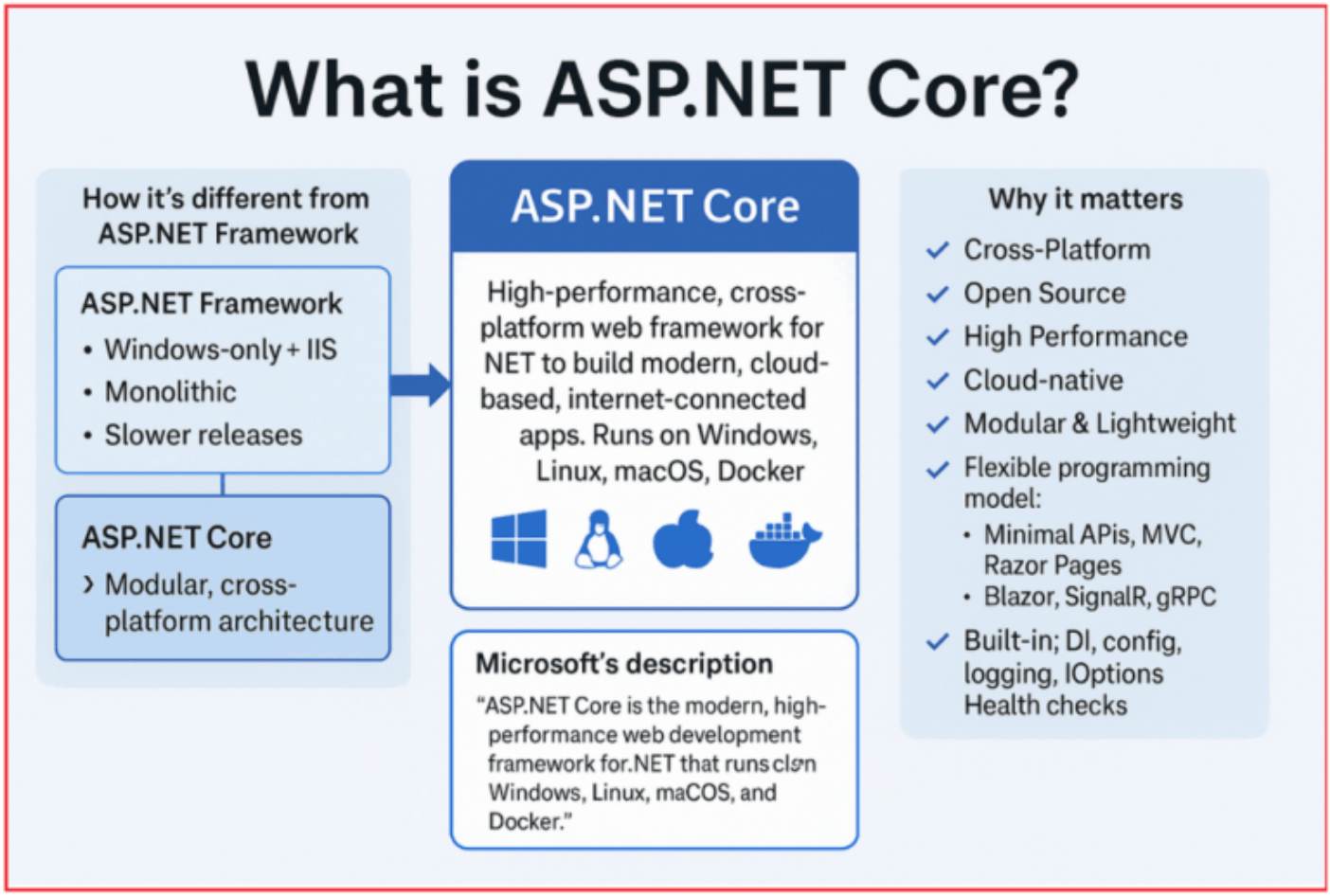
- **High Performance:** With its re-engineered request pipeline and **Kestrel web server**, ASP.NET Core consistently ranks among the **fastest web frameworks** in industry benchmarks.
- **Deployment:** Supports flexible hosting, IIS, Kestrel, Nginx, Apache, or self-hosted.
- **Lightweight & Modular:** Applications only load the required libraries via **NuGet packages**, resulting in smaller and faster code.
- **Cross-Platform:** Applications can now run on **Windows, Linux, macOS, and Docker**, expanding deployment options to a broader range of scenarios.
- **Cloud & Modern Development Ready**: ASP.NET Core was built with **cloud-first deployment, containerization, and microservices architectures** in mind.
- **Unified Programming Model** – It merges previously separate frameworks (MVC, Web API, Razor Pages) into a **single, consistent model** for building modern web applications.

In short, **ASP.NET Core represents the most significant evolution in Microsoft's web development ecosystem**, enabling developers to build **modern, scalable, and cloud-native applications** that surpass the limitations of the traditional ASP.NET Framework.

What is ASP.NET Core?

ASP.NET Core is a **Cross-Platform, Open-Source, High-Performance** web development framework created by Microsoft. It is used to build **modern, cloud-based, and internet-connected applications** that run on **Windows, Linux, macOS, and Docker**.

Unlike the earlier **ASP.NET Framework** (which was limited to Windows and IIS hosting), ASP.NET Core was built **from the ground up** with a **lightweight, modular, and flexible architecture** to meet the demands of modern development.



Microsoft's Description

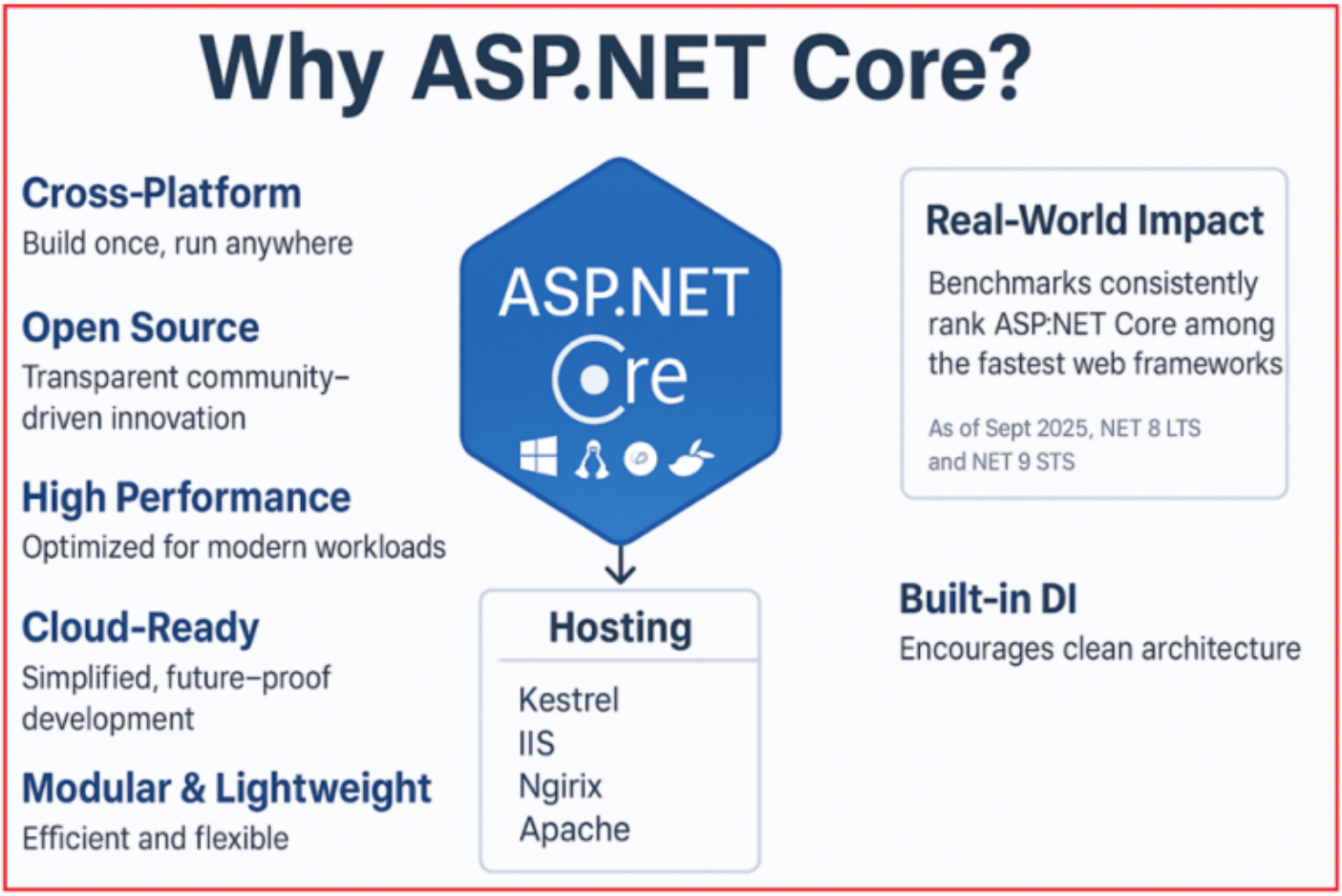
- ASP.NET Core is the modern, high-performance web development framework for .NET, running on Windows, Linux, macOS, and Docker.
- **ASP.NET** is a popular web development framework for building web applications on the **.NET Platform**.

- ASP.NET Core is the open-source version of ASP.NET that runs on macOS, Linux, and Windows. ASP.NET Core was first released in 2016 and is a redesign of earlier Windows-only versions of ASP.NET.

For more information, visit the official ASP.NET Core page: **https://dotnet.microsoft.com/en-us/learn/aspnet/what-is-aspnet-core**

Why ASP.NET Core?

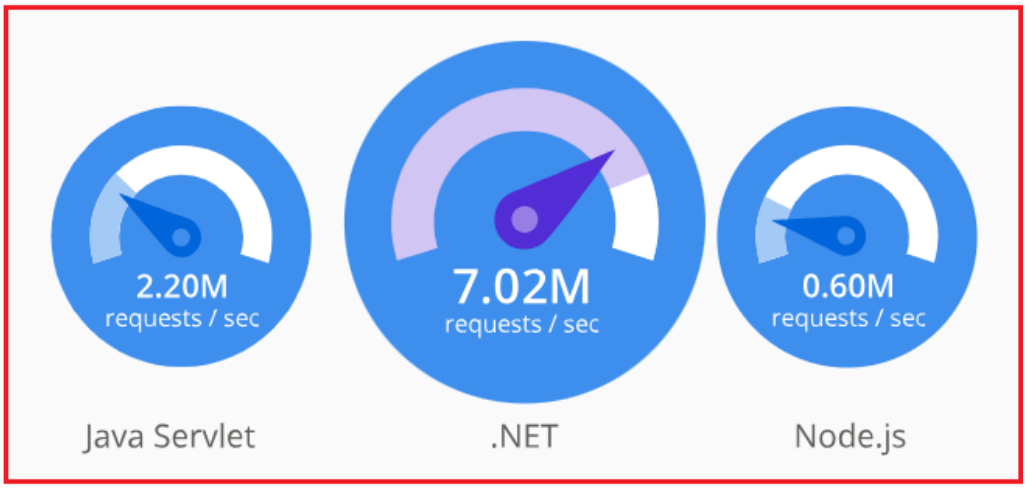
ASP.NET Core has quickly become the **preferred choice** for developers across industries. ASP.NET Core supports the latest development standards like REST APIs, minimal APIs, Razor Pages, Blazor (WebAssembly), and gRPC. Its popularity comes from several essential advantages. For a better understanding, please have a look at the following image:



High Performance

- ASP.NET Core is optimized for **Speed, Scalability, and Efficiency**.
- The **modular architecture** ensures that only required dependencies are loaded, making applications lighter and faster.
- Benchmarks consistently place ASP.NET Core among the **fastest web frameworks** worldwide.
- Ideal for **real-time, high-traffic, cloud-native applications** like e-commerce platforms, APIs, and microservices.

For a better understanding, please look at the following image, which is provided on the Official Microsoft Site:



Real-World Impact: Benchmarks demonstrate that ASP.NET Core outperforms traditional ASP.NET, Java Spring Boot, Node.js, and PHP in numerous scenarios.

Open Source

- .NET (ASP.NET Core) is an open-source, cross-platform framework maintained by Microsoft and the .NET community on **GitHub**.
- .NET has consistently ranked among the **top 30 most active open-source projects since 2017**, as tracked by the Cloud Native Computing Foundation.
- Developers can:
  - Inspect and understand the source code
  - Modify/customize it for specific needs
  - Contribute directly to framework development
- Backed by **100,000+ contributions** from **3,700+ companies**, ensuring continuous innovation.
- All aspects of .NET are open source, including class libraries, the runtime, compilers, languages, the ASP.NET Core web framework, Windows desktop frameworks, the Entity Framework Core data access library, and more.
- Microsoft and the community release frequent improvements, so you don’t have to wait months for bug fixes, new features or patches.

For more information, visit the official ASP.NET Core page: https://dotnet.microsoft.com/en-us/platform/open-source

Cross-Platform

- Designed from scratch as a **Cross-Platform Framework**.
- ASP.NET Core Applications can be developed and run on **Windows, Linux, macOS, and Docker** without code changes.
- Flexible hosting options:
  - Traditional: **IIS** (Windows)
  - Modern: **Nginx, Apache, or Kestrel** (Linux/macOS)
  - Cloud-native: **Docker/Kubernetes Containers**
- ASP.NET Framework applications were **Windows-only** and required **IIS hosting**.

Lightweight and Modular

- Applications are **built from smaller NuGet packages** instead of a heavy monolithic framework.
- Developers can **easily add/remove features**, keeping apps clean.
- Results in:
  - **Smaller application size**
  - **Improved performance**
  - **Faster development cycles**

Built-in Dependency Injection (DI)

- ASP.NET Core includes **first-class support for DI**, unlike the older ASP.NET Framework, where third-party libraries were needed.
- **Benefits:**
  - Simplifies **service and lifecycle management**

- Increases **testability and maintainability**
- Encourages **clean, decoupled architecture**

Cloud-Ready

- Designed with **cloud deployment and scaling** in mind.
- Provides configuration providers for **appsettings.json, environment variables, Azure Key Vault, and more**.
- Works seamlessly with **Microsoft Azure** and other cloud providers for scaling and CI/CD pipelines.
- Makes it an excellent choice for **microservices and cloud-native architectures**.

So, ASP.NET Core is more than just the next version of ASP.NET. It is a **complete redesign** of how web applications should be built in the **cross-platform, cloud-first era**.

- **Cross-Platform** → Build once, run anywhere.
- **Open Source** → Transparent, community-driven innovation
- **High Performance** → Optimized for modern workloads
- **Modular & Lightweight** → Efficient and flexible
- **Built-in DI & Cloud Ready** → Simplified, future-proof development

In short, **ASP.NET Core is Microsoft's flagship framework for building fast, scalable, and modern web applications**.

.NET Core Support Policy and Release Lifecycle:

Microsoft maintains a **transparent and predictable release schedule** for the .NET platform. This helps developers and organizations plan their **application upgrades, migrations, and support timelines** effectively.

Evolution of Naming

- Up to **.NET Core 3.1**, the framework was officially called **.NET Core**.
- From **.NET 5 (released in November 2020)** onward, Microsoft dropped the term *"**Core**"* and unified the name as **.NET**.
- Today, there is only **one .NET platform**, which covers all workloads:
  - ASP.NET Core (Web apps, APIs, Blazor)
  - .NET MAUI (mobile & desktop)
  - WPF & Windows Forms (desktop)
  - Cloud-native services & microservices
- This unification ensures **one SDK, one runtime, and one base class library (BCL)** across platforms.

Release Frequency

- Microsoft releases a **new major version of .NET every November**.
- Each release alternates between:
  - **Long-Term Support (LTS)** – **Even-numbered releases** (e.g., .NET 6, .NET 8)
  - **Standard-Term Support (STS)** – **Odd-numbered releases** (e.g., .NET 5, .NET 7)

Support Types

Long-Term Support (LTS)

- Even-numbered releases (e.g., **.NET 6, .NET 8**).
- Provide **3 years of free support and patches** (bug fixes, security updates, performance improvements).
- Recommended for **production environments** where **stability is critical**.
- Ideal for enterprises that cannot upgrade frequently.

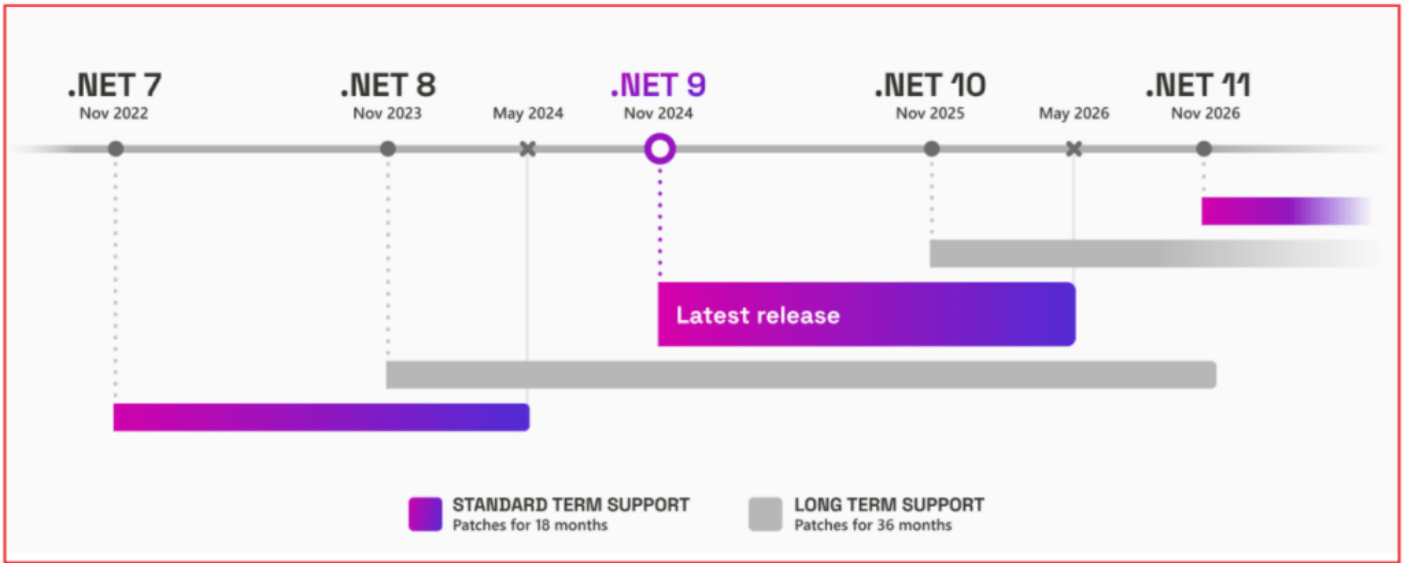
Standard-Term Support (STS)

- Odd-numbered releases (e.g., **.NET 5, .NET 7**).
- Provide **18 months of support**.
- Recommended for developers and organizations that:
  - Want to **adopt the latest features early**
  - Can **upgrade more frequently**
  - Work in **fast-moving projects or cloud environments**

Quality of Releases

- **Both LTS and STS releases are of the same quality.**
- The **only difference** is the **length of support**.
- After the support period ends, that version will no longer receive updates, which may expose applications to **security vulnerabilities** if not upgraded.

For a better understanding, please have a look at the following image:



Why This Matters for Developers

- Choosing the **right release type** (LTS vs STS) depends on project needs:
  - **Enterprise applications** → Prefer **LTS** for stability and long-term maintenance.
  - **Agile/startup projects** → Can adopt **STS** to take advantage of the latest features.
- Regular updates ensure applications remain **secure, stable, and performant**.

For more information, visit the official ASP.NET Core page: https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core

Why No .NET Core Version 4?

When Microsoft moved from the **old .NET Framework** to the **new .NET Core platform**, they wanted to make a **clean break** in versioning to avoid confusion.

Avoiding Confusion with .NET Framework 4.x

- At the time of **.NET Core 3.1** (released in December 2019), the **.NET Framework 4.x** series was still widely used.
- If Microsoft had named the next version **“.NET Core 4”**, many developers would mistakenly assume it was an **upgrade to .NET Framework 4.x**, when in reality, it was a **different product line**.
- Example of confusion: Developers might think **.NET Framework 4.8 → .NET Core 4.0** was a direct upgrade path (which it was not).

Unification Under “.NET 5 and Beyond”

- To create a **single unified platform**, Microsoft decided to **skip version 4 entirely**.
- After .NET Core 3.1, the next release was directly called **.NET 5 (Nov 2020)**.

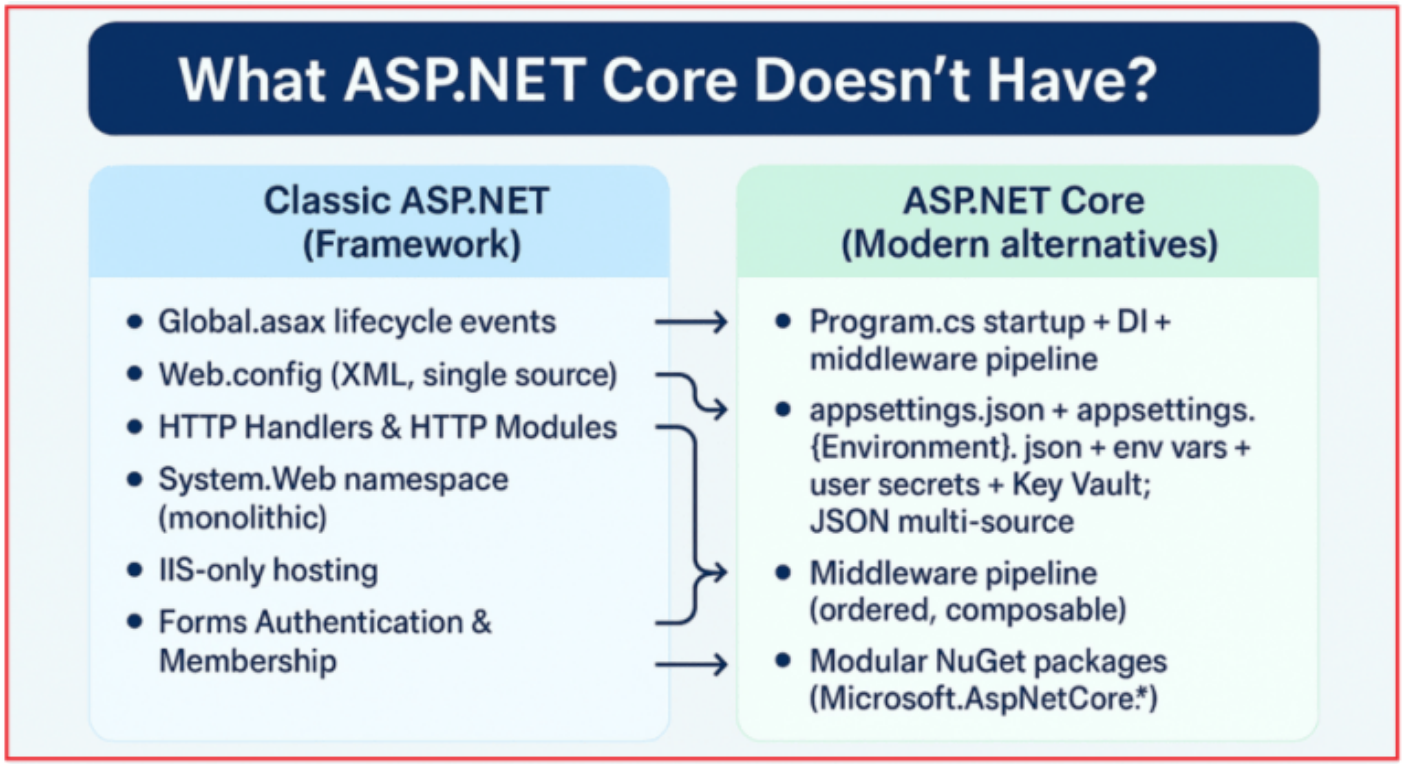
- From .NET 5 onwards, the word *"**Core**"* was dropped, and Microsoft moved towards **one .NET platform** for all workloads (desktop, mobile, web, cloud, IoT).

Aligning with Future Vision

- By skipping 4, Microsoft emphasized that **.NET 5+ is the future of .NET**, not a continuation of the old .NET Framework.
- This naming also clearly marked the **end of ".NET Framework" as a product line** (in maintenance mode) and the **beginning of unified .NET**.

What the ASP.NET Core Doesn't Have?

ASP.NET Core was designed as a **modern, lightweight, and modular framework**. To achieve this, Microsoft removed or **replaced many features** from the traditional ASP.NET Framework that were tightly coupled to **Windows, IIS, or System.Web**. For a better understanding, please have a look at the following image:



Global.asax File (Application Lifecycle Events)

- **ASP.NET Framework**:
  - The Global.asax file was used to handle **application-level events** (e.g., Application_Start, Session_Start, Application_End).
- **ASP.NET Core**:
  - No Global.asax.
  - Application startup is configured using the **Program.cs** file (and previously Startup.cs).

- Developers configure **middleware, services (via Dependency Injection), and the request pipeline** here, rather than hardcoding lifecycle events.

Web.config File (Configuration Settings)

- **ASP.NET Framework**:
  - Relied on a single **Web.config** file (XML format) for application settings, connection strings, and IIS configurations.
- **ASP.NET Core**:
  - Configuration is **flexible and multi-source**, handled via:
    - appsettings.json
    - appsettings.{Environment}.json (e.g., appsettings.Development.json)
    - Environment variables
    - Command-line arguments
    - User secrets (for development)
    - Azure Key Vault or other external providers
  - JSON-based config makes it **lighter, easier to read, and environment-specific**.

HTTP Handlers & HTTP Modules

- **ASP.NET Framework**:
  - Custom request handling was done with **HTTP Handlers** (for endpoints) and **HTTP Modules** (for pipeline events).
  - These were tied to the System.Web assembly and IIS.
- **ASP.NET Core**:
  - Replaced by the **Middleware Pipeline**.
  - Middleware components can be added in **any order** to handle cross-cutting concerns like authentication, routing, error handling, and logging.
  - More **flexible, composable, and cross-platform** than the old model.

System.Web Namespace

- **ASP.NET Framework**:
  - Depended heavily on the System.Web namespace, which was **large, monolithic, and Windows-only**.
- **ASP.NET Core**:
  - The System.Web namespace has been **removed**.
  - Instead, ASP.NET Core uses **lighter, modular NuGet packages** that developers add only when needed.

IIS Dependency

- **ASP.NET Framework**:

- Applications could run **only on IIS** (Internet Information Services).
- **ASP.NET Core**:
  - Runs on the **Kestrel web server** (cross-platform, built-in).
  - Can be hosted on **IIS, Nginx, Apache, Docker containers, or directly as a self-hosted service**.

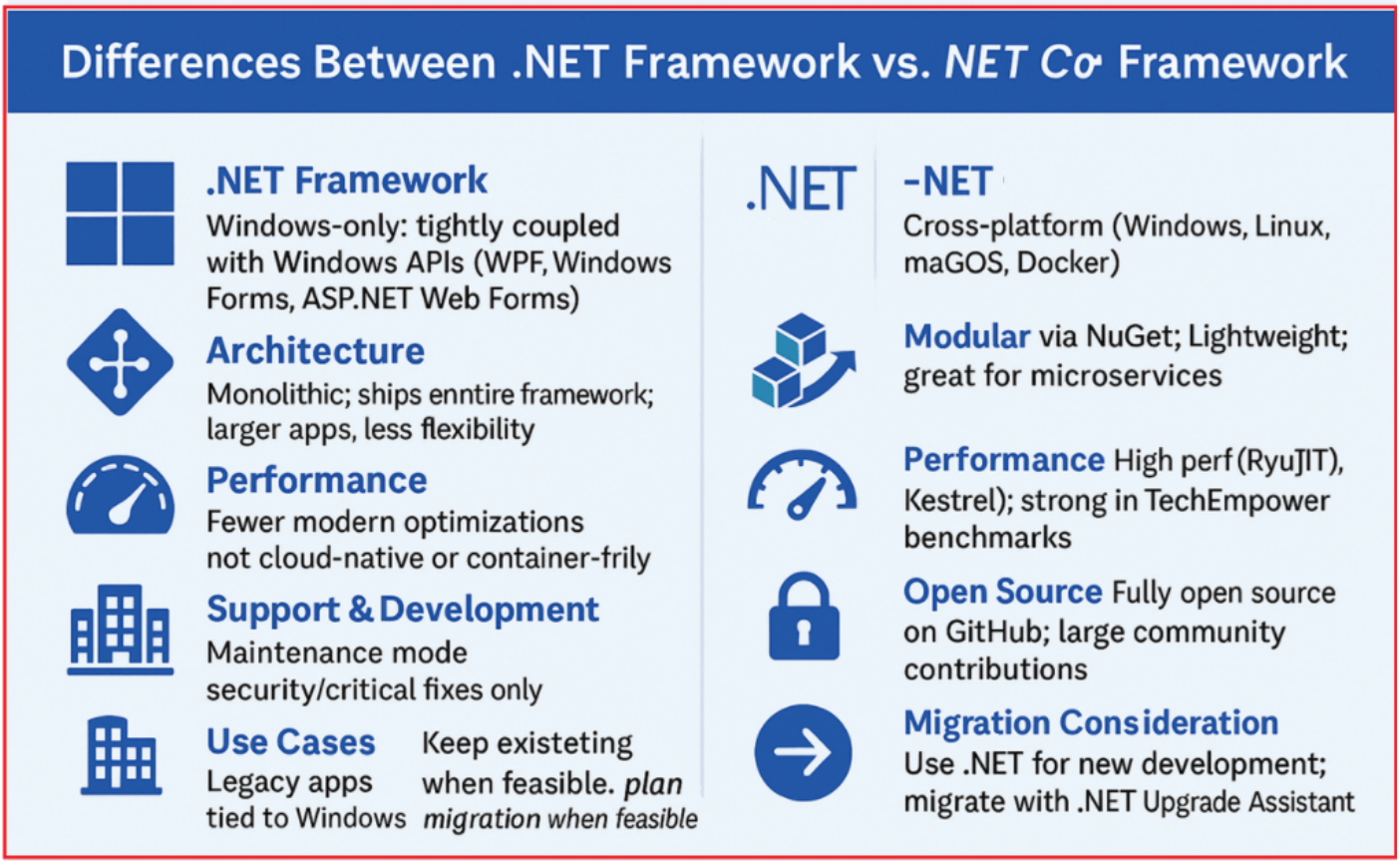
Forms Authentication & Membership

- **ASP.NET Framework**:
  - Had built-in **Forms Authentication, Membership, and Roles API**.
- **ASP.NET Core**:
  - Provides **ASP.NET Core Identity** (modern, extensible authentication/authorization system).
  - Supports **cookies, JWT, OAuth2, OpenID Connect, and external login providers** (Google, Facebook, Microsoft, etc.).

In short, **ASP.NET Core removes heavy, tightly coupled features** of the old ASP.NET Framework. It replaces them with **lightweight, flexible, and cross-platform alternatives**, making it far better suited for **modern cloud-native applications**.

Differences Between .NET Framework vs .NET Core Framework

The **.NET Framework** and **.NET Core** (now unified as just **.NET** since .NET 5) are both software development platforms created by Microsoft. While they share some similarities, they differ significantly in **platform support, architecture, performance, development status, and use cases**.



.NET Framework

1. **Platform**
   - Works **only on Windows**.
   - Tightly coupled with Windows APIs (e.g., WPF, WinForms, ASP.NET Web Forms).
2. **Architecture**
   - **Monolithic**: Ships with all libraries by default, even if not needed.
   - Results in a **larger application size** and **less flexibility**.
3. **Performance**
   - Limited optimizations compared to .NET Core.
   - Designed in the early 2000s, not optimized for today's **cloud-native, containerized, high-performance workloads**.
4. **Support and Development**
   - Currently in **maintenance mode**.
   - Receives only **security updates and critical fixes**.
   - **No new features** are being added (the last major version was **.NET Framework 4.8.1** in 2022).
5. **Open Source**
   - Closed-source (though some parts, like the Roslyn compiler and WCF client libraries, were open-sourced later).
   - Core runtime remains proprietary.
6. **Use Cases**
   - Still used in **large enterprise applications,** tightly integrated with Windows.

- Ideal for **legacy apps** that rely on:
  - Windows-specific tech (e.g., **WPF, Windows Forms**)
  - **COM components** or Windows-only APIs
- Maintains strong support in enterprise IT systems.

.NET Core (Now Unified .NET 5+)

1. **Platform**
   - **Cross-Platform**: Runs on **Windows, Linux, macOS, and Docker**.
   - Supports deployment in **cloud-native and containerized environments**.
2. **Architecture**
   - **Modular**: Applications include only the **NuGet packages** they need.
   - Lightweight, flexible, and optimized for **microservices**.
3. **Performance**
   - **High-performance runtime and JIT compiler (RyuJIT)**.
   - Built-in **Kestrel web server** for lightning-fast HTTP processing.
   - Frequently tops industry benchmarks (TechEmpower).
4. **Support and Development**
   - Actively developed with **yearly releases** (every November).
   - Alternates between **LTS (Long-Term Support, 3 years) and STS (Short-Term Support, 18 months)** versions.
   - Includes constant **performance improvements and new features**.
5. **Open Source**
   - Fully open-source and hosted on **GitHub**
   - Contributions from **100,000+ developers** and **3,700+ companies**.
6. **Use Cases**
   - Ideal for **modern applications**, such as:
     - Web APIs & Microservices
     - Cloud-native applications (Docker, Kubernetes)
     - Cross-platform tools and services
     - High-performance web apps (Blazor, MVC, Razor Pages)
   - Scales well for both **enterprise** and **startup projects**.

In short:

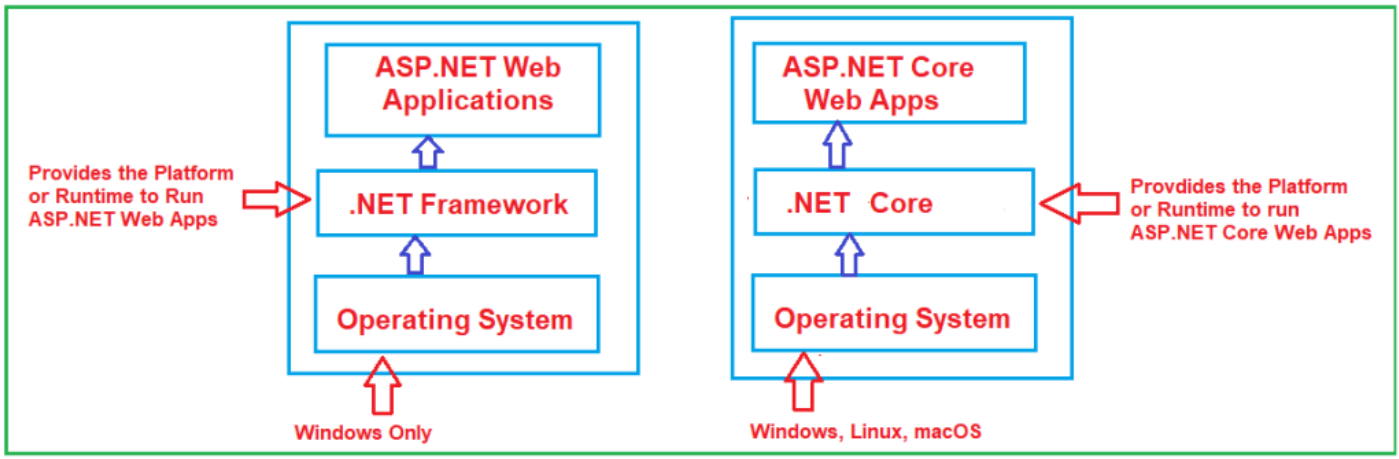
- **.NET Framework** → Legacy, Windows-only, in maintenance mode. Best for maintaining existing enterprise apps that depend on Windows-specific technologies.
- **.NET Core (.NET 5+)** → Modern, cross-platform, modular, high-performance, and cloud-ready. Best for building new applications and future-ready solutions.

Migration Consideration

- Applications built on **.NET Framework** still work and will continue to receive **security support**.
- However, for **new development**, Microsoft **recommends .NET (Core)** since it is **future-proof** and designed for **modern architectures**.
- Migration tools, such as the **.NET Upgrade Assistant,** help developers migrate projects from .NET Framework to .NET Core/.NET 6+.

.NET Core (.NET) vs ASP.NET Core:

A common source of confusion among beginners is the relationship between **.NET Core (.NET)** and **ASP.NET Core**. While they are closely related, they are **not the same thing**. Their relationship is similar to the older **.NET Framework vs ASP.NET** distinction. For a better understanding, please have a look at the following diagram:



What is .NET (.NET Core)?

.NET (formerly .NET Core) is the **runtime and base development platform**. It provides the **execution environment, runtime libraries, compilers, and tools** required to build and run applications across multiple platforms.

Key Points about .NET:

- **Cross-Platform Runtime**: Supports Windows, Linux, macOS, and Docker.
- **Application Types**: Powers many workloads beyond web apps, including:
  - Console applications
  - Desktop applications (via **WinForms, WPF, and .NET MAUI**)
  - Mobile applications (via **.NET MAUI / Xamarin**)
  - Cloud-native microservices
  - IoT and AI/ML integrations

- **SDK vs Runtime**:
  - **.NET Runtime** → Required to **run applications**.
  - **.NET SDK (Software Development Kit)** → Required to **develop and build applications** (includes runtime, compilers, and CLI tools).
- **Latest Stable Version**: **.NET 8 (LTS, released November 2023)**.

In short: **.NET is the foundation.**

What is ASP.NET Core?

ASP.NET Core is the **web application framework** that runs on top of .NET. It is specifically designed for building:

- Web Applications (MVC, Razor Pages)
- REST APIs / Web APIs
- Real-time apps using SignalR
- Single Page Applications (SPA) backends
- Blazor applications (WebAssembly & Server)
- gRPC services and Minimal APIs

Key Points about ASP.NET Core:

- **Open-Source & Cross-Platform** → Not limited to Windows and IIS; can run on Linux, macOS, Docker.
- **Hosting**: Runs on the **Kestrel web server** (cross-platform, high-performance) and can be reverse-proxied via **IIS, Nginx, or Apache**.
- **Dependency Injection**: Built-in DI support from the ground up.
- **Configuration**: Uses appsettings.json, environment variables, and providers (not Web.config).
- **Unified Model**: Merges MVC + Web API concepts into one framework.
- **Latest Stable Version**: **ASP.NET Core 8 (same as .NET 8)**.

In short, **ASP.NET Core is the web layer that uses .NET to run.**

Relationship Between .NET and ASP.NET Core

- **.NET provides the runtime**, tools, and base libraries.
- **ASP.NET Core is built on top of .NET** and is one of the workloads that uses the runtime.
- Without .NET, ASP.NET Core cannot run; however, .NET can exist independently of ASP.NET Core (e.g., for console or desktop applications).

Analogy:

- Think of **.NET** as the **operating system of a phone**, and
- **ASP.NET Core** is a **specific app (browser)** running on that OS.

Versioning

- There is **no separate versioning** for ASP.NET Core.
- It always aligns with the .NET runtime version.
- Example:
- .NET 6 → ASP.NET Core 6
- .NET 7 → ASP.NET Core 7
- .NET 8 → ASP.NET Core 8

What are the different flavors of the ASP.NET Core Framework?

ASP.NET Core is a **modular**, **cross-platform**, and **high-performance** framework for building modern web applications. It supports various "flavors" (or application models/frameworks) to suit different types of apps. Below are the **primary flavors** of the ASP.NET Core framework:

1. ASP.NET Core MVC

- Traditional server-side rendered web applications.
- **Key Features**:
  - Model-View-Controller pattern.
  - Razor Views for dynamic HTML rendering.
  - Strong support for RESTful routing.
- **Best For**: Enterprise apps, admin panels, and SEO-friendly web pages.

2. ASP.NET Core Web API

- RESTful API development for SPAs, mobile apps, or microservices.
- **Key Features**:
  - JSON-based data exchange (default is System.Text.Json).
  - Stateless HTTP services.
  - Attribute-based routing and filters.
- **Best For**: Backend APIs, microservices, third-party integrations.

3. ASP.NET Core Razor Pages

- **Use Case**: Simplified web apps with page-focused development.
- **Key Features**:
  - Page-centric model (each .cshtml file represents a page).
  - Encourages separation of concerns (UI logic in .cshtml.cs).
  - Minimal routing setup.
- **Best for**: Internal tools, simple web applications, and forms-based pages.

4. ASP.NET Core Blazor

The Blazor brings **C# to the browser**, replacing JavaScript in many cases.

Blazor Server

- **Execution**: Server-side (via SignalR connection).
- **Best for**: Intranet apps, low-latency apps, and apps requiring access to server-side resources.
- **Pros**: Small download size, SEO-friendly.
- **Cons**: Relies on a constant server connection.

Blazor WebAssembly (WASM)

- **Execution**: Runs completely in the browser via WebAssembly.
- **Best For**: Client-side apps, SPAs with offline support.
- **Pros**: Fully client-side, no server dependency.
- **Cons**: Larger download size, limited .NET APIs in the browser.

5. ASP.NET Core Minimal APIs (from .NET 6+)

- **Use Case**: Lightweight APIs with minimal boilerplate.
- **Key Features**:
  - No need for controllers or a startup class.
  - Ideal for microservices and small apps.
- **Best For**: Quick prototyping, microservices, serverless functions.

6. ASP.NET Core SignalR

- **Use Case**: Real-time web functionality.
- **Key Features**:
  - WebSockets, Server-Sent Events, Long Polling (fallbacks).
  - Built-in support for persistent connections.
- **Best For**: Chat apps, live dashboards, notifications.

7. ASP.NET Core gRPC

- **Use Case**: High-performance, contract-first communication using Protobuf.
- **Key Features**:
  - HTTP/2-based RPC.
  - Strong typing with .proto files.
- **Best For**: Microservices, internal service-to-service calls.

ASP.NET Core is more than just an upgrade; it's a **complete redesign** to meet the needs of modern development. It offers **high performance**, **cross-platform support**, **a modular design**, and **built-in features** such as dependency injection and cloud readiness. Whether you're creating small web apps or large enterprise systems, ASP.NET Core is built to handle it all. With Microsoft's strong support and a growing developer community, ASP.NET Core is clearly the **future of web development** on the .NET platform.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.